TIP COMPUTER ENGINEERING
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	09/20/2026	**DATE SUBMITTED:**	09/22/2026
NAME:	Dela Cruz, Jared Isaiah P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 8.1 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 8-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp.

A.1. Shell Sort

General Algorithm
The general algorithm for shell sort is given below.

```
shellSort(array, size)
  for interval i <- size/2n
    down to 1 for each
    interval "i" in array
        sort all the elements at
interval "i" end shellSort
```

A.2. Merge Sort

General Algorithm
The general pseudo-code for the merge sort technique is given below.

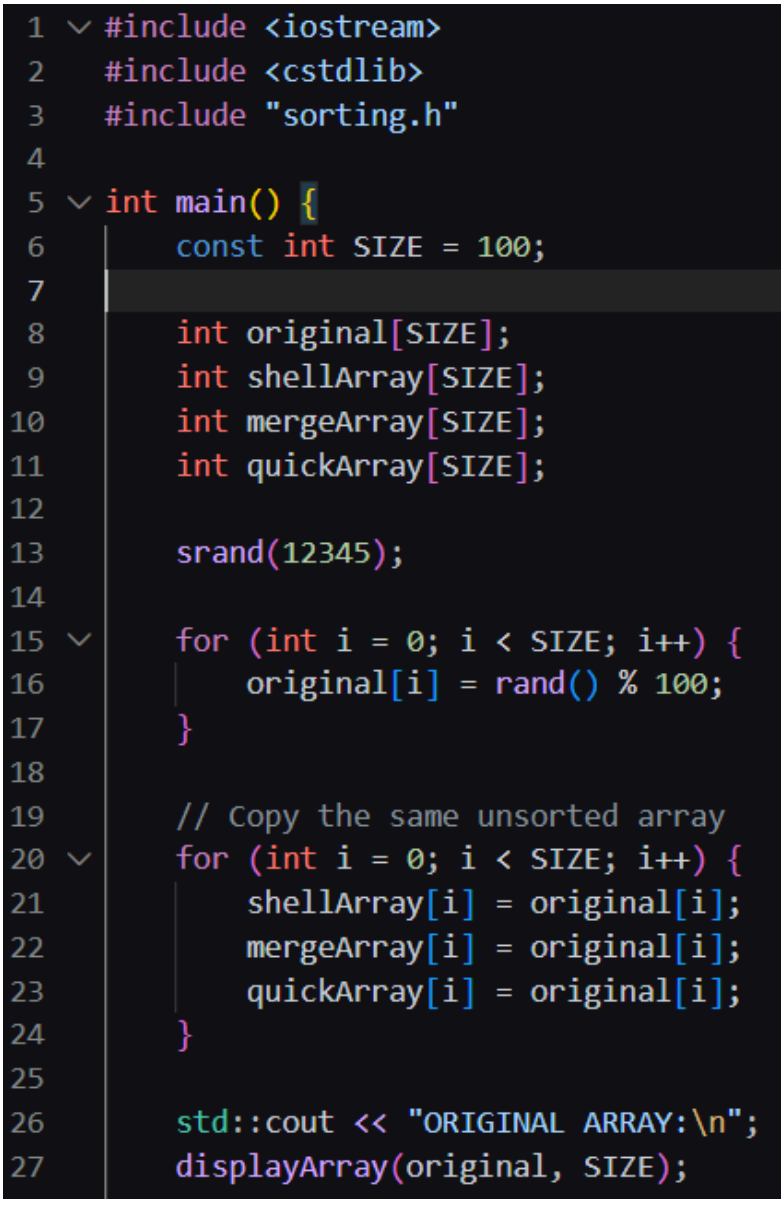
```
Declare an array Arr of length N If N=1, Arr is already sorted

If N>1,
       Left = 0, right = N-1
       Find middle = (left + right)/2
       Call merge_sort(Arr,left,middle) =>sort first half recursively
```

```
Call merge_sort(Arr,middle+1,right) => sort second half recursively
Call merge(Arr, left, middle, right) to merge sorted arrays in above
steps. Exit
```

A.3. Quick Sort

```
quicksort(A, low, high) begin
Declare array A[N] to be sorted
low = 1st element; high = last element; pivot if(low < high)
begin
pivot = partition (A,low,high); quicksort(A,low,pivot-1)
quicksort(A,pivot+1,high)
end
```

Code Screenshot	

```cpp
#include <iostream>
#include <cstdlib>
#include "sorting.h"

int main() {
    const int SIZE = 100;

    int original[SIZE];
    int shellArray[SIZE];
    int mergeArray[SIZE];
    int quickArray[SIZE];

    srand(12345);

    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 100;
    }

    // Copy the same unsorted array
    for (int i = 0; i < SIZE; i++) {
        shellArray[i] = original[i];
        mergeArray[i] = original[i];
        quickArray[i] = original[i];
    }

    std::cout << "ORIGINAL ARRAY:\n";
    displayArray(original, SIZE);
```

	``` ORIGINAL ARRAY: 84 64 95 25 28 5 77 13 72 4 59 0 43 32 34 76 66 16 75 49 18 25 18 84 36 85 1 88 28 69 68 12 28 13 31 52 7 36 27 25 92 52 31 88 90 84 57 25 14 47 70 58 56 0 60 20 7 86 0 37 68 30 64 2 ``` ``` 81 70 0 16 24 63 2 74 74 89 9 24 46 7 4 37 18 80 57 60 93 80 48 47 87 30 4 91 60 51 39 67 62 54 93 40 ```
Observations	The displayArray() function prints every item in the array one by one without changing them. It is useful for checking the array's contents before and after sorting. It runs in O(n) time and uses O(1) extra space since it checks each item once.

Table 8-1. Array of Values for Sort Algorithm Testing

Code Screenshot	```
	template <typename T>
	void shellSort(T arr[], int arrSize) {
	    for (int interval = arrSize / 2; interval > 0; interval /= 2) {
	
	        for (int i = interval; i < arrSize; i++) {
	            T temp = arr[i];
	            int j = i;
	
	            while (j >= interval && arr[j - interval] > temp) {
	                arr[j] = arr[j - interval];
	                j -= interval;
	            }
	
	            arr[j] = temp;
	        }
	    }
	}
	``
	``
	shellSort(shellArray, SIZE);
	std::cout << "\n\nSHELL SORT:\n";
	displayArray(shellArray, SIZE);
	``

	SHELL SORT: 0 0 0 0 1 2 2 4 4 4 5 7 7 7 9 12 13 13 14 16 16 18 18 18 20 24 24 25 25 25 25 27 28 28 28 67 68 68 69 70 70 72 74 74 75 76 77 80 80 81 84 84 84 85 86 87 88 88 89 90 91 92 93 93 95 30 30 31 31 32 34 36 36 37 37 39 40 43 46 47 47 48 49 51 52 52 54 56 57 57 58 59 60 60 60 62 63 64 64 66
Observations	Shell Sort improves on insertion sort by comparing items that are farther apart rather than just side by side. It uses a gap size that gets smaller step by step until it reaches 1, which helps organize the list early on. The algorithm repeatedly shrinks the gap and arranges items within those smaller sub-groups. Depending on the gap sequence chosen, its worst-case time complexity is O(n^2).

Table 8-2. Shell Sort Technique

Code Screenshot	

```
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    int leftSize = middle - left + 1;
    int rightSize = right - middle;

    T* leftArray = new T[leftSize];
    T* rightArray = new T[rightSize];

    for (int i = 0; i < leftSize; i++) {
        leftArray[i] = arr[left + i];
    }

    for (int i = 0; i < rightSize; i++) {
        rightArray[i] = arr[middle + 1 + i];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while (i < leftSize && j < rightSize) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        }
        else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }
```

```
while (i < leftSize && j < rightSize) {
    if (leftArray[i] <= rightArray[j]) {
        arr[k] = leftArray[i];
        i++;
    }
    else {
        arr[k] = rightArray[j];
        j++;
    }
    k++;
}

while (i < leftSize) {
    arr[k] = leftArray[i];
    i++;
    k++;
}

while (j < rightSize) {
    arr[k] = rightArray[j];
    j++;
    k++;
}

delete[] leftArray;
delete[] rightArray;
```

```
template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left < right) {
        int middle = left + (right - left) / 2;

        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);

        merge(arr, left, middle, right);
    }
}
```

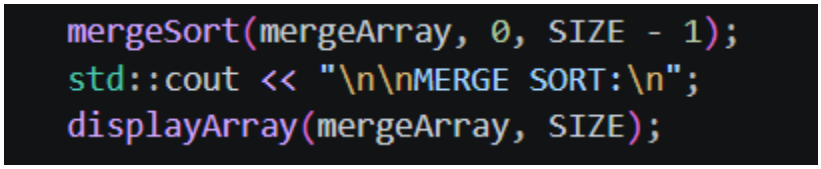
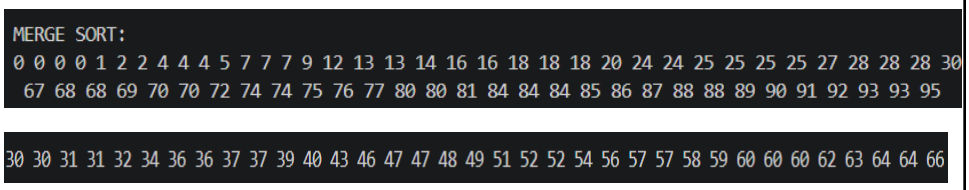
	 
Observations	Merge Sort follows a divide-and-conquer strategy by repeatedly splitting the array in half until each piece has only one item. It then combines those pieces back together in the correct order. The function calls itself recursively on both halves and uses a merge() helper function to combine them. This helper function compares items from two sorted sections, builds a sorted list in a temporary array, and copies the results back to the original array. Its time complexity is always $O(n \log n)$ in both the best and worst cases.

Table 8-3. Merge Sort Algorithm

Code Screenshot

```
template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);

        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}
```

```
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);

    return i + 1;
}
```

```
quickSort(quickArray, 0, SIZE - 1);
std::cout << "\n\nQUICK SORT:\n";
displayArray(quickArray, SIZE);
```

```
QUICK SORT:
0 0 0 0 1 2 2 4 4 4 5 7 7 7 9 12 13 13 14 16 16 18 18 18 20 24 24 25 25 25 25 27 28 28 28 30
 67 68 68 69 70 70 72 74 74 75 76 77 80 80 81 84 84 84 85 86 87 88 88 89 90 91 92 93 93 95
```

```
30 31 31 32 34 36 36 37 37 39 40 43 46 47 47 48 49 51 52 52 54 56 57 57 58 59 60 60 60 62 63 64 64 66
```

Observations	Quick Sort is another divide-and-conquer algorithm. It picks a "pivot" value, moves smaller numbers to its left and larger numbers to its right, and then recursively sorts each side until the sections have one or no elements left. The partition() helper function handles choosing the pivot, rearranging the items, and returning the pivot's final position. quickSort() then uses that position to divide and sort the remaining sections. Its time complexity depends on the chosen pivot, but the worst-case scenario is O(n^2).

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

The left and right sublists created by Quick Sort's partition() method do not have to be sorted recursively. Instead, you can use a different algorithm to sort those sublists after partitioning.

Example: [7, 4, 9, 3, 8, 2,]

```
int left[] = {4, 3, 2};
int right[] = {7, 9, 8};

int leftSize = sizeof(left) / sizeof(left[0]);
int rightSize = sizeof(right) / sizeof(right[0]);

std::cout << "\n\nProblem 1\n";
std::cout << "Left sublist before sorting: ";
displayArray(left, leftSize);
std::cout << "\nRight sublist before sorting: ";
displayArray(right, rightSize);

insertionSort(left, leftSize);
insertionSort(right, rightSize);

std::cout << "\n\nLeft sublist after Insertion Sort: ";
displayArray(left, leftSize);
std::cout << "\nRight sublist after Insertion Sort: ";
displayArray(right, rightSize);

std::cout << "\n\nFinal result with pivot 6: ";
displayArray(left, leftSize);
std::cout << "6 ";
displayArray(right, rightSize);
```

```
Problem 1
Left sublist before sorting: 4 3 2
Right sublist before sorting: 7 9 8

Left sublist after Insertion Sort: 2 3 4
Right sublist after Insertion Sort: 7 8 9

Final result with pivot 6: 2 3 4 6 7 8 9
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Quick Sort is usually the fastest of these algorithms, provided a good pivot is chosen. However, Merge Sort is better if you need predictable performance every time. Both run in O(N \log N) average time because they divide the array into about log N levels and process N items at each level. By comparison, Shell Sort can drop to O(N^2), while basic methods like Bubble, Selection, and Insertion Sort consistently take O(N^2) time in their average and worst cases.

3. CONCLUSIONS

Through this activity, I gained a practical understanding of how Shell Sort, Merge Sort, and Quick Sort operate and how different input data affects their efficiency. Counting the actual number of comparisons gave me a clear, real-world view of their performance beyond theoretical Big-O notation. Additionally, I learned that Quick Sort can integrate other sorting algorithms to handle its sublists after partitioning.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the assistance:

Tools Used: **ChatGPT, Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and code debugging and analysis. Also used for information gathering**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **AI was only used to assist me in accomplishing my tasks through grammar correction and code analysis. It was also used to gather more information about the different functions of the sorting algorithms that helped me arrive to the program I made.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.